모델 설계하기

국립금오공과대학교
시종욱

목차

- 개발 환경 구축
- 딥러닝 모델 정의
- 하이퍼 파라미터 설정
- 모델 학습

필요한 SW 목록

Python
Numpy
Pandas
Matplotlib/Seaborn
Scikit-Learn

Anaconda

GPU

TensorFlow

Colab

colab.research.google.com

로컬 환경 구축

- 그래픽카드, NVidia Cuda, CudNN, Python/Tensorflow 버전맞추기, 변수 경로설정...
- GPU를 구했다하더라도 이것을 딥러닝 개발을 하기위한 세팅을 하는데는 엄청나게 스트레스가 많이 생김

딥러닝 모델 정의

- 폐암 수술 환자의 생존율 예측

```
# 필요한 라이브러리
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# 실행할 때마다 같은 결과를 출력하기 위한 seed
np.random.seed(3)
tf.random.set_seed(3)

# 환자 기록을 불러옵니다.
df = pd.read_csv("ThorarixSurgery.csv", header=None)
data_set = df.to_numpy()

# X, Y 분리
X = data_set[:, 0:17].astype(np.float32)
Y = data_set[:, 17].astype(np.float32)
```

딥러닝 모델 정의

- 폐암 수술 환자의 생존율 예측

```
# 딥러닝 구조 정의
model = Sequential()
model.add(Dense(30, input_shape=(17,), activation='relu'))
model.add(Dense(1, activation='sigmoid'))

# 딥러닝 실행
model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy', tf.keras.metrics.Recall(name='recall')]
)

model.fit(X, Y, epochs=100, batch_size=10, verbose=1)
```

딥러닝 모델 정의

- 데이터 불러오기
  - ➢ 17가지의 속성, 그리고 하나의 클래스(정답)

속성 / 클래스

항목 \ 줄	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
1	293	1	3.8	2.8	0	0	0	0	0	0	12	0	0	0	1	0	62	0
2	1	2	2.88	2.16	1	0	0	0	1	1	14	0	0	0	1	0	60	0
3	8	2	3.19	2.5	1	0	0	0	1	0	11	0	0	1	1	0	66	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
470	447	8	5.2	4.1	0	0	0	0	0	0	12	0	0	0	0	0	49	0

딥러닝 모델 정의

- 데이터 불러오기
  - ➢ 종양의 유형, 폐활량, 호흡 곤란 여부, 고통 정보, 기침, 흡연, 천식 여부 등
  - ➢ 클래스: 생존 여부 (1: 생존, 0: 사망)

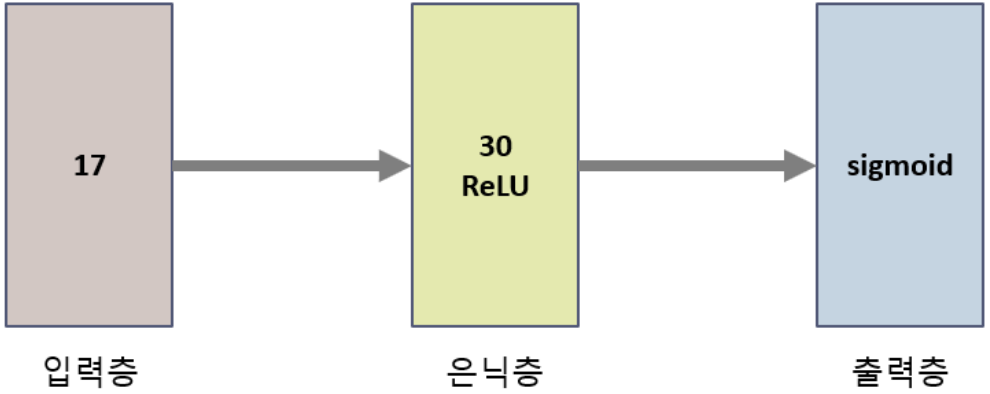
속성 클래스

항목 \ 줄	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
1	293	1	3.8	2.8	0	0	0	0	0	0	12	0	0	0	1	0	62	0
2	1	2	2.88	2.16	1	0	0	0	1	1	14	0	0	0	1	0	60	0
3	8	2	3.19	2.5	1	0	0	0	1	0	11	0	0	1	1	0	66	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
470	447	8	5.2	4.1	0	0	0	0	0	0	12	0	0	0	0	0	49	0

딥러닝 모델 정의

- 모델

```
# 딥러닝 구조 정의
model = Sequential()
model.add(Dense(30, input_shape=(17,), activation='relu'))
model.add(Dense(1, activation='sigmoid'))
```



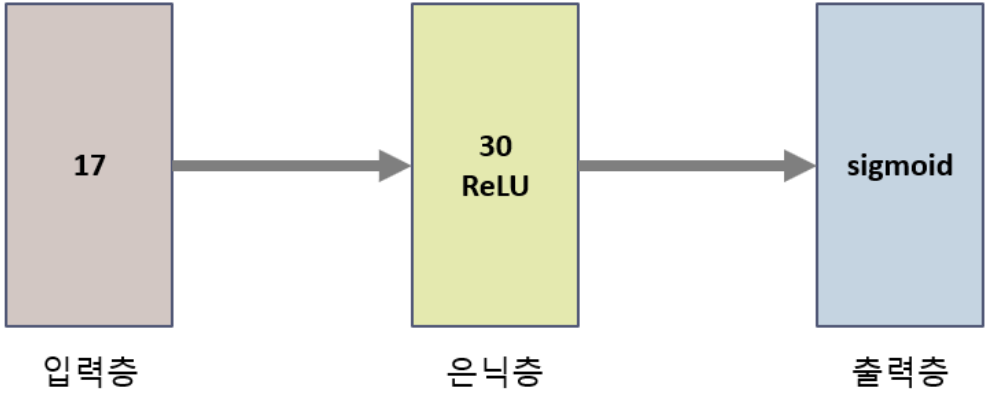
딥러닝 모델 정의

- 모델

```
# 딥러닝 구조를 정의합니다.
model = Sequential([
        Dense(30, input_shape=[17], activation='relu'),
        Dense(1, activation='sigmoid')
])
```

```
input_shape=(17,)
```

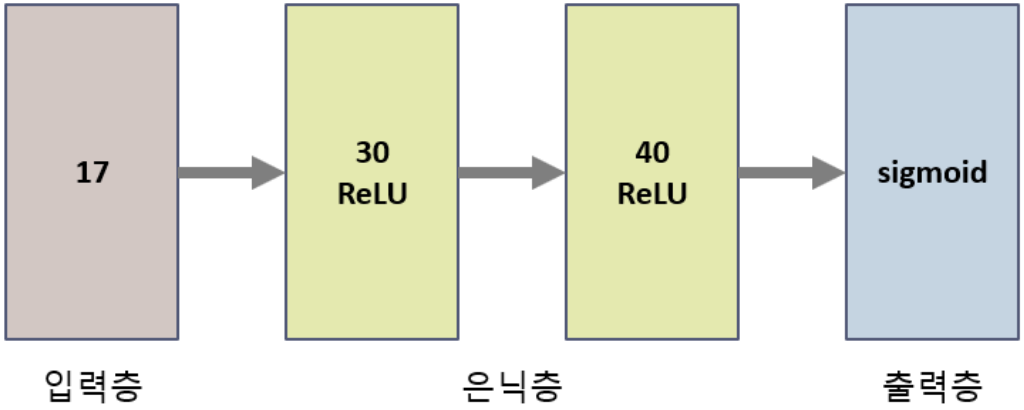
```
input_shape=[17]
```



딥러닝 모델 정의

- 모델

```
# 딥러닝 구조를 정의합니다.
model = Sequential()
model.add(Dense(30, input_dim=(17,), activation='relu'))
model.add(Dense(40, input_dim=(30,), activation='relu'))
model.add(Dense(1, activation='sigmoid'))
```



하이퍼 파라미터 설정

- 모델 환경 설정
  - 정의한 모델에 여러가지 환경 설정
  - 비용 함수(loss function), 최적화 방법(optimizer), 모델 성능 평가 기준(metrics)

```
model.compile(loss='binary_crossentropy', optimizer='adam’,
    metrics=['accuracy', tf.keras.metrics.Recall()])
```

하이퍼 파라미터 설정

- 모델 환경 설정

https://www.tensorflow.org/api_docs/python/tf/keras/losses

평균 제곱 계열	mean_squared_error	평균 제곱 오차 계산: mean(square(yt - yo))
	mean_absolute_error	평균 절대 오차(실제 값과 예측 값 차이의 절댓값 평균) 계산: mean(abs(yt - yo))
	mean_absolute_percentage_error	평균 절대 백분율 오차(절댓값 오차를 절댓값으로 나눈 후 평균) 계산: mean(abs(yt - yo)/abs(yt) (단, 분모 ≠ 0)
	mean_squared_logarithmic_error	평균 제곱 로그 오차(실제 값과 예측 값에 로그를 적용한 값의 차이를 제곱한 값의 평균) 계산: mean(square((log(yo) + 1) - (log(yt) + 1)))
교차 엔트로피 계열	categorical_crossentropy	범주형 교차 엔트로피(일반적인 분류)
	binary_crossentropy	이항 교차 엔트로피(두 개의 클래스 중에서 예측할 때)

하이퍼 파라미터 설정

- 모델 환경 설정

https://www.tensorflow.org/api_docs/python/tf/keras/optimizers

고급 경사 하강법	개요	효과	케라스 사용법
확률적 경사 하강법 (SGD)	랜덤하게 추출한 일부 데이터를 사용해 더 빨리, 자주 업데이트를 하게 하는 것	속도 개선	keras.optimizers.SGD(lr = 0.1) 케라스 최적화 함수를 이용합니다.
모멘텀 (Momentum)	관성의 방향을 고려해 진동과 폭을 줄이는 효과	정확도 개선	keras.optimizers.SGD(lr = 0.1, momentum = 0.9) 모멘텀 계수를 추가합니다.
네스테로프 모멘텀 (NAG)	모멘텀이 이동시킬 방향으로 미리 이동해서 그레이디언트를 계산. 불필요한 이동을 줄이는 효과	정확도 개선	keras.optimizers.SGD(lr = 0.1, momentum = 0.9, nesterov = True) 네스테로프 옵션을 추가합니다.
아다그라드 (Adagrad)	변수의 업데이트가 잦으면 학습률을 적게 하여 이동 보폭을 조절하는 방법	보폭 크기 개선	keras.optimizers.Adagrad(lr = 0.01, epsilon = 1e - 6) 아다그라드 함수를 사용합니다. ※ 참고: 여기서 epsilon, rho, decay 같은 파라미터는 바꾸지 않고 그대로 사용하기를 권장하고 있습니다. 따라서 lr, 즉 learning rate(학습률) 값만 적절히 조절하면 됩니다.

하이퍼 파라미터 설정

- 모델 환경 설정 https://www.tensorflow.org/api_docs/python/tf/keras/optimizers

고급 경사 하강법	개요	효과	케라스 사용법
알엠에스프롭 (RMSProp)	아다그라드의 보폭 민감도를 보완한 방법	보폭 크기 개선	keras.optimizers.RMSprop(lr = 0.001, rho = 0.9, epsilon = 1e - 08, decay = 0.0) 알엠에스프롭 함수를 사용합니다.
아담(Adam)	모멘텀과 알엠에스프롭 방법을 합친 방법	정확도와 보폭 크기 개선	keras.optimizers.Adam(lr = 0.001, beta_1 = 0.9, beta_2 = 0.999, epsilon = 1e - 08, decay = 0.0)

```
# 객체 생성하여 사용
sgd = optimizers.SGD(lr=0.01, decay=1e-6, momentum=0.9, nesterov=True)
model.compile(loss='mean_squared_error', optimizer=sgd)

# 기본 설정 사용
model.compile(loss='mean_squared_error', optimizer='sgd')
```

하이퍼 파라미터 설정

- 모델 환경 설정
  - accuracy, mse, AUC, recall 등
  - https://www.tensorflow.org/api_docs/python/tf/keras/metrics

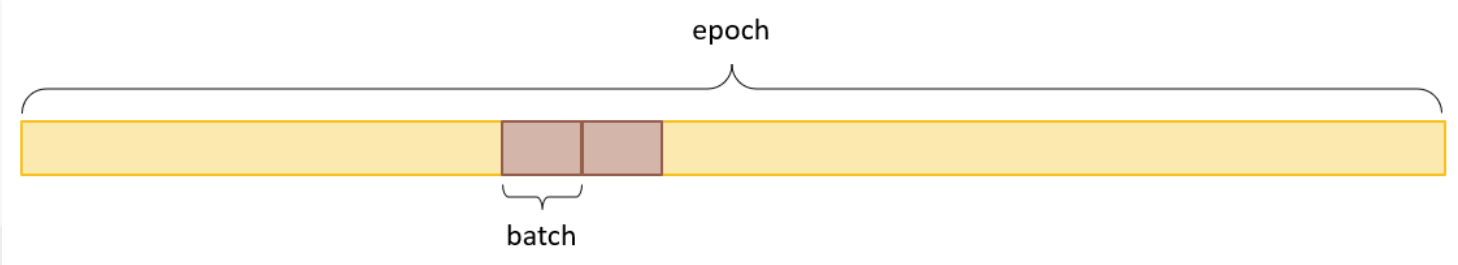
```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['ac
curacy', 'AUC', tf.keras.metrics.Recall()])
```

```
Epoch 1/100 47/47 [==============================] - 0s 1ms/step - loss: 0.8142 - accuracy: 0.8064 -
auc: 0.4989 - recall: 0.0143
Epoch 2/100 47/47 [==============================] - 0s 1ms/step - loss: 0.5553 - accuracy: 0.8149 -
auc: 0.4867 - recall: 0.0000e+00
Epoch 3/100 47/47 [==============================] - 0s 1ms/step - loss: 0.4792 - accuracy: 0.8404 -
auc: 0.5211 - recall: 0.0286
. . .
```

모델 학습

- 학습 실행하기
  - 학습 데이터를 사용하여 정의한 모델에 대한 학습 수행
  - 반복 횟수(epochs)와 배치 사이즈(batch_size) 지정

```
model.fit(X, Y, epochs=100, batch_size=10)
```



실습 1

- 은닉층 노드 수 바꾸기
  - ➢ 현재 은닉층은 Dense(30)으로 설정되어 있다. 이를 10, 20, 50으로 바꾸고 결과를 비교하시오.
    - ✓ accuracy가 어떻게 변하는가?
    - ✓ recall이 어떻게 변하는가?
    - ✓ 노드 수가 너무 적거나 많을 때 어떤 차이가 있는가?

실습 2

- 은닉층 1개 더 추가하기
  - ➢ 기존 모델은 은닉층이 1개이다. 10개 노드를 가지는 은닉층 하나를 추가하여 결과를 확인하시오.
    - ✓ 은닉층을 추가했을 때 성능이 좋아지는가?
    - ✓ 학습 시간이 얼마나 늘어나는가? (대략적)

실습 3

- 은닉층 여러 개 추가하기
  - ➢ 더 깊은 신경망을 구성하기 위해 각각의 유닛의 개수가 (64-32-16-1)로 설정하고 결과를 확인하시오.
    - ✓ 층이 깊어질수록 항상 성능이 좋아지는가?

실습 4

- 활성화 함수 바꿔보기
  - ➢ 입력층의 activation function을 relu, sigmoid, tanh로 바꿔보고 성능을 비교해보시오.
    - ✓ 어떤 활성화 함수가 가장 좋은 결과를 보이는가?
    - ✓ 학습 속도의 차이가 있는가? (대략적)

실습 5

- Loss 함수 바꿔보기
  - ➢ 현재 손실함수는 binary_crossentropy이다. mean_squared_error로 변경하여 결과를 확인하시오.
    - ✓ 분류 문제에서 어떤 loss가 더 적절한지 분석하시오.

실습 6

- Batch Size 함수 바꿔보기
  - ➢ Batch_size를 5, 10, 20, 50으로 바꿔서 학습한 후 결과를 확인하시오.
    - ✓ batch_size가 작을수록 어떠한 차이가 있는가?
    - ✓ 큰 batch_size와 작은 batch_size의 장단점은 무엇인가?

참고 문헌

- Intro to Deep Learning

  https://www.kaggle.com/learn/intro-to-deep-learning